

## High-Scale E-Commerce Flash Sale System

### Problem Statement:

Design a scalable e-commerce architecture capable of handling massive concurrent purchase requests for limited-stock products while maintaining inventory consistency, payment reliability and correct order processing.

TEAM MEMBERS: DEEPIGA R S  
NETHRA HARINI K  
NIVETHINI R V  
SAKTHI KAMALAM G

DESIGN FIRST • SCALE SAFELY • NEVER OVERSELL

# SALESTORM: WHEN DEMAND EXPLODES

## THE PROBLEM



### Massive Traffic Spike

Up to 10,000 customers send purchase requests simultaneously.



### Duplicate Requests

Users may click "Buy Now" multiple times, but only one reservation should be created.



### Payment Failures

Failed payments must not create orders, and reserved inventory should be released.



### Inventory Consistency

Use atomic reservation and concurrency control to prevent overselling.



### Scalability

System should scale horizontally to handle peak demand without downtime.



### Limited Inventory

Only 100 units are available, which must never be oversold.



### Reservation Expiry

Unpaid or timed-out reservations must be released automatically to make stock available.



### Service Failures

System should handle database, payment gateway or order service failures gracefully with retries.



### Idempotent Processing

Repeated requests must not create duplicate payments or duplicate orders.



### Observability

Monitor request rate, latency, error rate, inventory and payment failures in real time.



## OUR SOLUTION



### ABSORB

Handle traffic spikes intelligently

- CDN / WAF / Load Balancer
- Rate limiting
- Horizontal scaling
- Caching of product & sale data



### RESERVE

Protect limited inventory

- Atomic inventory reservation
- Concurrency control
- Idempotency
- Reservation expiry
- No negative inventory



### COMPLETE

Reliable payment and order processing

- Idempotent payments
- Retry and timeout handling
- Event-driven order creation
- Consistent order lifecycle
- Failure recovery and notifications



Scale the traffic. Control the contention. Ensure every successful purchase is correct.



## WHY SALESTORM WORKS

### Massive Traffic

Handles 10,000 concurrent buyers during flash-sale spikes.

### Limited Stock

Atomic reservation ensures inventory never goes below zero.

### No Duplicates

Idempotency prevents duplicate reservations, payments and orders.

### Payment Safe

Failed or timed-out payments release reserved inventory.

### Failure Recovery

Queues, retries and reconciliation recover failed workflows.

### Scales Fast

Stateless services, caching and load balancing support horizontal scaling

## USE CASES



Flash-sale events



Limited-stock drops



High-demand launches



Peak shopping events



## INNOVATION & UNIQUENESS



### Concurrency-Safe Inventory

Atomic reservation prevents overselling when thousands of buyers compete for limited stock



### Idempotent Transactions

Every critical request uses an idempotency key, preventing duplicate reservations **payments and orders.**



### Event-Driven Recovery

Durable events, retries and reconciliation ensure failed payment or order workflows can recover safely



### Controlled Contention

Only the inventory boundary controls access to scarce stock instead of allowing every request to compete directly.



### Reservation Lifecycle

AVAILABLE → RESERVED → PAYMENT → CONFIRMED / RELEASED



### Elastic Scalability

Stateless services, caching, load balancing and queues allow the platform to scale horizontally during traffic spikes.

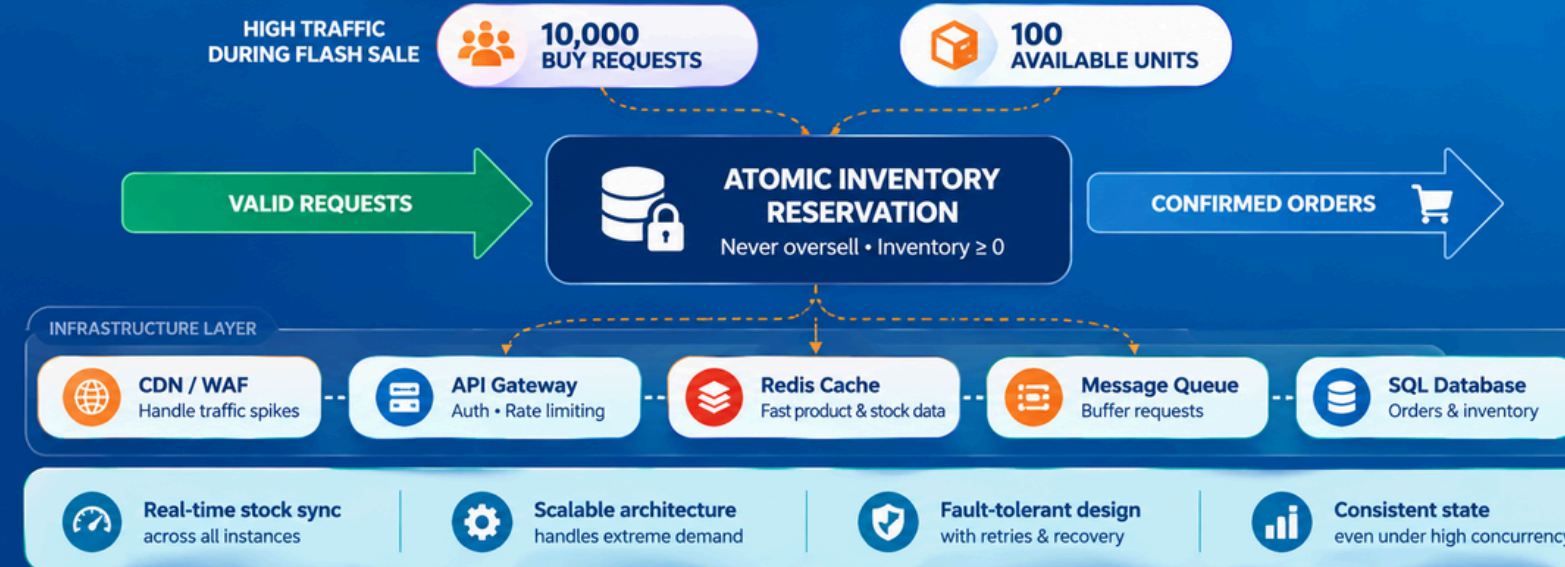
“The system does not try to process everyone immediately — it controls contention at the inventory boundary.”

# TECHNICAL APPROACH

## CUSTOMER / EDGE

- 1. REQUEST**  
Customer clicks “Buy Now” during the flash sale.
- 2. VALIDATE**  
API Gateway authenticates, validates and applies rate limiting.
- 3. DEDUPLICATE**  
Idempotency key prevents duplicate purchase attempts.
- 4. ROUTE**  
Load balancer distributes requests across stateless services.

## 4. RESERVE: ONE CONTROLLED INVENTORY BOUNDARY



## CHECKOUT / ORDER

- 5. Payment**  
Idempotent payment prevents duplicate charges.
- 6. Recover**  
Timeouts trigger retry, reconciliation or stock release.
- 7. Confirm**  
Successful payment creates the final order.

## TRAFFIC MANAGEMENT

- Rate Limiting**  
Controls sudden request bursts during flash-sale spikes.
- Load Balancing**  
Distributes traffic across multiple stateless service instances.
- Caching**  
Serves product and sale data quickly, reducing backend load.
- Queueing**  
Buffers requests and smooths traffic during extreme demand.

## SECURITY & RELIABILITY

- Idempotent**  
Repeated requests cannot create duplicate orders or payments.
- Atomic**  
Inventory updates happen safely under concurrent requests.
- Consistent**  
Stock never falls below zero, even during the flash sale.
- Recoverable**  
Retries and reconciliation handle payment or service failures.

## TECHNOLOGIES USED

- 1 Frontend:** React.js • HTML • CSS
- 2 Backend:** Node.js • Express.js • REST API
- 3 Database:** MongoDB • Redis Cache
- 4 Traffic Management:** API Gateway • Load Balancer • Rate Limiting
- 5 Reliability:** Message Queue • Idempotency • Retry
- 6 Monitoring & Observability:** Logs • Metrics • Error Tracking

10,000 CONCURRENT BUYERS • 100 UNITS • ZERO OVERSELLING

“Built for the rush. Designed for consistency.”

# FEASIBILITY AND VIABILITY



## FEASIBILITY

Proven tech, scalable and implementable



### MODERN WEB STACK

- React.js frontend, Node.js backend
- REST APIs, MongoDB, Redis



### HANDLES HIGH TRAFFIC

- Load balancing, caching, queueing
- Designed for flash sale spikes



### SCALABLE ARCHITECTURE

- Stateless services
- Can scale horizontally on cloud/VPS



### SECURITY BUILT-IN

- Authentication, rate limiting
- Idempotency, secure payments



### BACKED BY RESEARCH

- Real-world flash sale case studies
- Best practices for concurrency control



## PLANNED FOR THE REAL WORLD

Every flash sale risk is addressed with an engineering solution

HIGH DEMAND.  
CONCURRENT USERS.  
ZERO OVERSELLING.

1



### Massive Traffic

Thousands of users click BUY NOW at the same time

2



### Validate & Limit

Authenticate users, apply rate limiting and deduplicate requests

3



### Check Inventory

Atomic reservation using Redis to prevent over-selling

4



### Process Payment

Idempotent payment to avoid duplicate charges

5



### Create Order

Confirm order only after payment success

6



### Complete

Send notification and move to fulfilment



## TECH STACK & INFRASTRUCTURE



**CDN / WAF**  
Handle traffic spikes



**API Gateway**  
Auth & Rate limiting



**Redis Cache**  
Fast product & inventory data



**Message Queue**  
Buffer requests during high load



**MongoDB**  
Users, orders & products



**Monitoring**  
Logs, metrics & alerts



## VIABILITY

Practical, scalable, ready for impact



### BETTER USER EXPERIENCE

- Faster, fair and smooth checkout
- Reduces failures during flash sales



### COST EFFECTIVE

- Built with open-source technologies
- Can be deployed on cloud/VPS



### SCALABLE & MODULAR

- Can handle more users and products
- New features can be added easily



### SECURE & RELIABLE

- Prevents duplicate orders
- Maintains consistent inventory



### REAL-WORLD IMPACT

- Applicable to e-commerce platforms
- Improves fairness and trust in flash sales

1



### FRONTEND – USER REQUEST

- Product browsing & real-time stock view
- Click BUY NOW during flash sale
- Sends request with idempotency key

2



### BACKEND – PROCESSING

- Validate user & apply rate limiting
- Atomic inventory reservation (Redis)
- Process payment (idempotent)
- Create order and update database

3



### DEPLOYMENT & OPERATIONS

- Deploy on cloud/VPS
- Auto-scaling and monitoring
- Handle failures with retries
- Ensure zero overselling

“10,000 buyers. 100 units. Zero overselling. One resilient architecture.”

# IMPACT AND BENIFITS



“Fast enough for the spike. Safe enough for every order.”

# RESEARCH AND REFERENCES

## RESEARCH & INSPIRATION

*Building a reliable e-commerce flash sale system for high-concurrency environments*

**VLDB 2019**

### Handling High-Concurrency Flash Sales in E-Commerce Platforms

Discusses inventory reservation, traffic control and system design patterns used by large-scale e-commerce platforms.

[Source](#)

**ACM 2020**

### Idempotent APIs for Reliable Distributed Transactions

Explores idempotency mechanisms to prevent duplicate orders and payments in high-load systems.

[Source](#)

**IEEE 2021**

### Scalable Architectures for E-Commerce Flash Sales

Studies load balancing, caching and queue-based designs to handle massive traffic spikes.

[Source](#)



### HIGH-CONCURRENCY PROBLEM 10,000 Buyers for 100 Units

Simulates real-world flash sale scenarios with extreme concurrent traffic and limited inventory.

[GitHub](#)



### BACKEND SYSTEM Node.js + Express.js

REST APIs with rate limiting, idempotency, inventory reservation and payment simulation.

[GitHub](#)



### DATABASE & CACHE MongoDB + Redis

Stores product, user and order data. Redis used for atomic inventory reservation and fast access.

[GitHub](#)



### TRAFFIC MANAGEMENT Load Balancer & Rate Limiting

Handles traffic spikes, prevents abuse and ensures fair access to the sale event.

[GitHub](#)



### MESSAGE QUEUE Redis Queue

Buffers high traffic requests and processes orders asynchronously to maintain stability.

[GitHub](#)



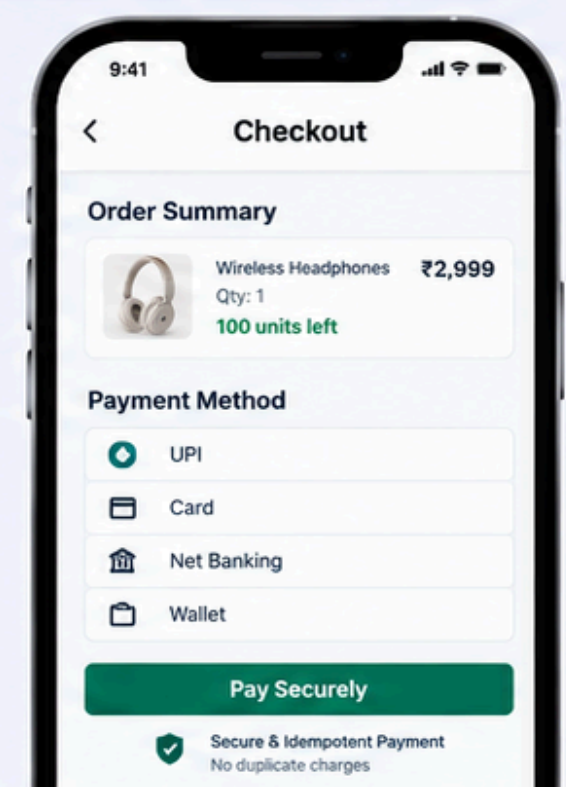
### RELIABILITY & MONITORING Idempotency + Logging

Prevents duplicate orders, handles failures with retries and maintains consistent inventory.

[GitHub](#)



## PROTOTYPE GLIMPSES: SALESTORM DEMO



## KEY HIGHLIGHTS



**10,000**  
Concurrent Buyers



**100**  
Available Units



**0**  
Overselling



**Idempotent**  
Payments



**Real-time**  
Inventory Sync



**Scalable**  
Architecture

**“10,000 buyers can compete for 100 units — without overselling, duplicate orders or lost transactions.”**

